ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN HỆ ĐIỀU HÀNH

BÁO CÁO THỰC HÀNH

MÔN HỆ ĐIỀU HÀNH

Lab 01

Xv6 User Programs: File System

Lớp học phần: *[Mã lớp / Tên lớp]*

Sinh viên: 24120438 – Trần Nguyên Tân

24120023 – Nguyễn Thành Bảo

24120040 – Dương Hoài Đức

Giảng viên: *[ThS./TS. Tên giảng viên]*

Mục lục

Tóm tắt báo cáo **2**

1 Thông tin thành viên và phân công công việc **3**

2 Môi trường thực hiện và kiểm thử **3**

3 Kiến thức nền tảng và thiết kế hệ thống **4**

3.1 Các system call được sử dụng 4

3.2 Quản lý tài nguyên 5

3.3 Xử lý lỗi và thông báo kết quả 5

4 Các chương trình đã thực hiện **6**

4.1 sleep 6

4.2 pingpong 7

4.3 primes 8

4.4 cp 10

4.5 tree 11

4.6 du 12

4.7 diff 13

5 Kết quả kiểm thử tổng hợp **15**

5.1 Môi trường và cách thực hiện 15

5.2 Kết quả theo từng chương trình 15

5.3 Nhật ký thực thi và phạm vi kết luận 16

6 Khó khăn gặp phải và giải pháp khắc phục **17**

7 Kết luận và hướng phát triển **17**

Tài liệu tham khảo **18**

Tóm tắt báo cáo

Báo cáo này trình bày kết quả thực hiện, thiết kế kỹ thuật và quá trình kiểm thử toàn diện các chương trình tiện ích không gian người dùng (*User Programs*) trong khuôn khổ bài thực hành **Lab 01 (Hệ điều hành xv6 – Kiến trúc RISC-V)**. Mục tiêu trọng tâm của bài thực hành là tiếp cận kiến trúc phân tầng người dùng/nhân (*User/Kernel Space*), vận dụng các system call cơ bản, cơ chế quản lý vòng đời tiến trình, truyền thông liên tiến trình (IPC) và các thao tác truy xuất hệ thống tệp tin (*File System*).

Trong suốt quá trình thực hiện, nhóm đã phân tích yêu cầu, xây dựng mã giả và hiện thực thành công toàn bộ **07 chương trình tiện ích**:

- **Phân nhóm Tiến trình & IPC:**
  - sleep: Tạm dừng tiến trình theo chu kỳ đồng hồ (*ticks*) của hệ thống.
  - pingpong: Hiện thực cơ chế giao tiếp 2 chiều đồng bộ giữa tiến trình cha và con thông qua cặp đường ống (*pipes*).
  - primes: Xây dựng thuật toán sàng số nguyên tố Eratosthenes đồng thời (*Concurrent Prime Sieve*) theo mô hình pipeline đa tiến trình kết nối liên tiếp từ 2 đến 280.
- **Phân nhóm Tệp tin & Duyệt hệ thống tệp:**
  - cp: Sao chép dữ liệu tập tin an toàn qua bộ đệm (*buffer streaming*), xử lý kiểm tra trùng tệp nguồn/đích.
  - tree: Duyệt đệ quy cây thư mục hiển thị phân cấp trực quan, hỗ trợ giới hạn độ sâu (-L) và lọc thư mục (-d).
  - du: Ước tính dung lượng sử dụng đĩa (*Disk Usage*) theo byte, hỗ trợ tính tổng nhanh (-s) và liệt kê chi tiết tệp (-a).
  - diff: So sánh nội dung hai tệp tin theo từng dòng, xử lý so khớp EOF và chế độ thông báo tóm tắt (-q).

Tóm tắt kết quả & Đánh giá chung:

- **Tỷ lệ hoàn thành:** 7/7 chương trình (100% khối lượng yêu cầu).
- **Độ tin cậy & Quản lý tài nguyên:** Tất cả các chương trình đều được kiểm thử nghiêm ngặt trên trình giả lập QEMU, kiểm soát chặt chẽ việc giải phóng bộ mô tả tập tin (không rò rỉ File Descriptor), đóng đúng các đầu pipe thừa để kích hoạt tín hiệu EOF chính xác, và đồng bộ hóa triệt để bằng wait() nhằm loại bỏ hoàn toàn nguy cơ sinh tiến trình ma (*zombie process*).

> - **Kiểm thử tự động:** Đáp ứng đầy đủ các ca kiểm thử chức năng và trường hợp biên lỗi theo chuẩn kịch bản kiểm thử của môn học.

1. Thông tin thành viên và phân công công việc

STT	MSSV	Họ và tên	Nhiệm vụ phụ trách
1	24120438	Trần Nguyên Tân	sleep, pingpong, primes
2	24120023	Nguyễn Thành Bảo	cp, diff
3	24120040	Dương Hoài Đức	tree, du, tích hợp và kiểm thử chung

Phương thức phối hợp nhóm: *[Mô tả ngắn gọn quy trình làm việc: phân chia nhánh trên Git (feature branches), viết pull request, review code chéo và kiểm thử tích hợp trên môi trường chung.]*

2. Môi trường thực hiện và kiểm thử

- **Hệ điều hành chủ / máy ảo:** *[Ubuntu 24.04 LTS trên WSL2 (Windows 11) / Linux / macOS]*
- **Trình biên dịch (Toolchain):** *[riscv64-linux-gnu-gcc (GCC 12.x / 13.x)]*
- **Trình giả lập phần cứng:** *[QEMU emulator version 8.x.x (qemu-system-riscv64)]*
- **Kho mã nguồn & nhánh làm việc:** *[Nhánh main / commit SHA...]*
- **Quy trình biên dịch và khởi chạy:**
  - Dọn dẹp bản dựng: make clean
  - Biên dịch và khởi chạy máy ảo: make qemu
  - Chạy toàn bộ bài chấm điểm: make grade

Kết quả xác nhận môi trường thực thi:

```
$ riscv64-linux-gnu-gcc --version
riscv64-linux-gnu-gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0

$ qemu-system-riscv64 --version
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.2)
```

3. Kiến thức nền tảng và thiết kế hệ thống

Trong bài thực hành, pingpong và primes dùng tiến trình con để trao đổi và xử lý dữ liệu; cp, tree, du và diff làm việc với tệp và thư mục. Dù có chức năng khác nhau, các chương trình đều cần kiểm tra kết quả thao tác, đóng tài nguyên sau khi sử dụng và thông báo rõ khi không thể tiếp tục. Phần này trình bày những kiến thức chung được áp dụng trong mã nguồn.

3.1 Các system call được sử dụng

System call là cách chương trình yêu cầu nhân xv6 thực hiện các thao tác như mở tệp, tạo tiến trình hoặc đọc dữ liệu. Các system call chính trong bài được sử dụng như sau:

- **Tạo và chờ tiến trình:** fork() tạo tiến trình con. Giá trị trả về giúp chương trình phân biệt phần việc của cha và con: 0 ở tiến trình con, PID của con ở tiến trình cha, và giá trị âm nếu thất bại. Trong pingpong và primes, tiến trình cha dùng wait() để chờ con kết thúc và thu hồi thông tin tiến trình con.
- **Trao đổi dữ liệu qua pipe:** pipe() tạo một đường truyền một chiều, gồm đầu đọc và đầu ghi. pingpong dùng hai pipe cho hai chiều trao đổi; primes nối các tiến trình bằng pipe để chuyển những số còn lại sang bước sàng tiếp theo.
- **Mở và đóng tệp:** open() trả về một số nguyên dùng để truy cập tệp, gọi là bộ mô tả tệp. cp mở nguồn để đọc, còn đích được mở với các cờ tạo tệp, ghi và xóa nội dung cũ. Các chương trình gọi close() khi không còn dùng tệp hoặc một đầu pipe.
- **Đọc và ghi dữ liệu:** read() và write() làm việc với cả tệp lẫn pipe. Giá trị trả về cho biết số byte thực sự đã đọc hoặc ghi. Vì vậy, cp ghi đúng lượng dữ liệu vừa đọc được, kể cả phần cuối ngắn hơn bộ đệm; pingpong kiểm tra đủ một byte cho mỗi lượt trao đổi.
- **Lấy thông tin tệp:** fstat() cung cấp loại tệp, kích thước và thông tin nhận diện của tệp đang mở. cp dùng thông tin này để tránh sao chép một tệp lên chính nó; diff kiểm tra hai đầu vào là tệp thông thường. tree và du dùng stat() để lấy thông tin theo đường dẫn. Trong xv6, stat() là hàm thư viện kết hợp open(), fstat() và close(), không phải một system call riêng.
- **Tạm dừng và kết thúc:** sleep() tạm dừng tiến trình theo số tick đã nhập. getpid() lấy PID để in kết quả trong pingpong. exit() kết thúc chương trình

và trả mã trạng thái cho tiến trình cha.

3.2 Quản lý tài nguyên

Đóng đúng các đầu pipe. Sau fork(), cả cha và con đều có các bộ mô tả tệp đã mở trước đó. Mỗi tiến trình cần đóng những đầu pipe không thuộc phần việc của mình. Khi bên gửi đã đóng tất cả các đầu ghi và dữ liệu trong pipe đã được đọc hết, bên nhận mới nhận được dấu hiệu hết dữ liệu (EOF). Nếu còn giữ một đầu ghi không dùng, bên nhận có thể tiếp tục chờ dù không còn số nào được gửi. Trong primes, việc đóng đầu ghi sau khi chuyển hết các số giúp tiến trình kế tiếp kết thúc vòng đọc.

Chờ tiến trình con kết thúc. Trên luồng chạy bình thường, các tiến trình cha trong pingpong và primes gọi wait() sau khi hoàn thành phần gửi dữ liệu. Việc này giúp thu hồi thông tin của tiến trình con đã kết thúc, thay vì để con ở trạng thái chờ được thu hồi. Với primes, mỗi bước sàng chỉ tạo tiến trình kế tiếp khi có số cần chuyển tiếp, tránh tạo thêm tiến trình ở cuối chuỗi.

Thu dọn tệp và bộ nhớ. cp đóng cả tệp nguồn lẫn tệp đích sau khi sao chép hoặc khi gặp lỗi đọc, ghi. tree và du đóng thư mục ở từng lượt gọi đệ quy. diff giải phóng bộ nhớ chứa hai dòng và đóng hai tệp trước khi thoát. Một số nhánh lỗi trong pingpong và primes gọi exit() trực tiếp; nhân xv6 sẽ đóng các bộ mô tả tệp còn mở khi tiến trình kết thúc.

Chọn bộ đệm phù hợp. cp sao chép từng phần với bộ đệm 512 byte, không cần giữ toàn bộ tệp trong bộ nhớ. tree và du dùng bộ đệm đường dẫn 256 byte và kiểm tra độ dài trước khi nối tên mục con. Riêng diff cấp phát bộ đệm cho từng dòng, tăng kích thước khi cần và giải phóng vùng nhớ cũ sau khi thay thế. Cách này cho phép đọc dòng dài hơn kích thước ban đầu, trong giới hạn bộ nhớ có thể cấp phát.

3.3 Xử lý lỗi và thông báo kết quả

Kiểm tra tham số trước khi thực hiện. sleep yêu cầu đúng một tham số, không rỗng và chỉ gồm chữ số; cp yêu cầu hai đường dẫn. tree kiểm tra tham số đi kèm -L và từ chối độ sâu không dương. tree, du và diff cũng kiểm tra các tùy chọn được hỗ trợ. Khi cú pháp không hợp lệ, chương trình báo lỗi thay vì tiếp tục với tham số thiếu hoặc sai.

Kiểm tra kết quả thao tác. Những thao tác như open(), pipe(), fork() và fstat() có thể thất bại, nên chương trình cần kiểm tra trước khi dùng kết quả. Với thao tác đọc, giá trị dương là số byte nhận được, 0 là hết dữ liệu và giá trị âm là lỗi.

cp phân biệt hết tệp với lỗi đọc, đồng thời coi việc ghi thiếu byte là thất bại. diff kiểm tra lỗi đọc, cấp phát bộ nhớ và ghi nội dung dòng; hàm in dòng tiếp tục ghi phần còn lại nếu một lần ghi chưa hết dữ liệu.

Bảo vệ dữ liệu và chọn phạm vi dừng. cp kiểm tra tệp đích trước khi mở với cờ xóa nội dung cũ, đặc biệt là trường hợp nguồn và đích cùng trỏ đến một tệp. Với tree, nếu không lấy được thông tin một mục con thì chương trình báo lỗi rồi duyệt tiếp các mục khác. du trả về 0 cho nhánh không truy cập được, nên tổng in ra có thể chưa bao gồm toàn bộ dữ liệu khi có lỗi. Đây là cách xử lý hiện có trong mã nguồn, không phải cam kết rằng việc duyệt luôn đầy đủ.

Thông báo lỗi được ghi ra luồng lỗi chuẩn bằng fprintf(2, ...), thường kèm tên lệnh và đường dẫn liên quan để người dùng biết thao tác nào thất bại. Các lệnh thường dùng mã thoát 0 khi hoàn thành và 1 cho lỗi được phát hiện ở mức chương trình chính. Riêng diff dùng 0 khi hai tệp giống nhau, 1 khi có khác biệt và 2 khi có lỗi; việc hai tệp khác nhau là kết quả so sánh, không phải lỗi thực thi.

Việc kiểm tra lỗi chưa đồng đều ở mọi vòng đọc: primes, tree và du dừng vòng lặp khi không đọc đủ một phần tử nhưng chưa tách riêng trường hợp hết dữ liệu và lỗi đọc. Vì vậy, phần báo cáo chỉ mô tả các kiểm tra đã có, không xem đây là cơ chế xử lý lỗi đầy đủ cho mọi tình huống.

4. Các chương trình đã thực hiện

4.1 sleep

Mục tiêu.

- **Chức năng:** Tạm dừng tiến trình hiện tại trong số chu kỳ đồng hồ do ticks chỉ định.
- **Tham số:** Chấp nhận đúng một tham số bắt buộc theo cú pháp sleep ticks; ticks phải là một giá trị số hợp lệ.
- **Xử lý lỗi:** Phát hiện trường hợp thiếu, thừa hoặc sai định dạng tham số và thông báo lỗi phù hợp.

Ý tưởng triển khai. Lệnh sleep sử dụng cơ chế chờ của hệ điều hành thay vì tự đếm thời gian bằng một vòng lặp. Sau khi kiểm tra tham số không rỗng và chỉ chứa chữ số, chương trình chuyển giá trị nhập thành số tick rồi yêu cầu tạm dừng tiến trình. Trong thời gian đó, tiến trình không chiếm CPU để chờ, nên hệ thống có thể chạy các công việc khác. Hết khoảng chờ, tiến trình được

phép chạy tiếp và kết thúc. Thời lượng ở đây được tính theo tick của xv6, không phải theo giây.

Mã giả dưới đây mô tả các bước kiểm tra tham số và tạm dừng tiến trình trong hàm main().

sleep

```
1  PROCEDURE Main(arguments)
2      IF arguments do not contain exactly one value THEN
3          REPORT usage error; EXIT 1
4      END IF
5      value ← the supplied argument
6      IF value is empty or contains a non-digit character THEN
7          REPORT invalid duration; EXIT 1
8      END IF
9      ticks ← ConvertToInteger(value)
10     Suspend the current process for ticks clock ticks
11     EXIT 0
12 END PROCEDURE
```

Nhận xét & khó khăn: *[Ghi chú các trường hợp đặc biệt (ví dụ: ticks âm hoặc bằng 0).]*

4.2 pingpong

Mục tiêu.

- **Chức năng:** Minh họa giao tiếp hai chiều giữa tiến trình cha và tiến trình con bằng cách gửi “ping” và nhận phản hồi “pong”.
- **Tham số:** Không nhận tham số dòng lệnh; lệnh được thực thi trực tiếp bằng pingpong.
- **Kết quả:** In thông báo giao tiếp kèm PID của các tiến trình và kết thúc bình thường sau khi trao đổi hoàn tất.

Ý tưởng triển khai. Hai tiến trình trao đổi qua hai pipe, một pipe cho chiều cha gửi sang con và một pipe cho chiều phản hồi. Tiến trình cha gửi một byte để bắt đầu lượt trao đổi. Khi nhận được byte này, tiến trình con in thông báo nhận ping, gửi lại byte đó rồi kết thúc. Cha chờ con hoàn tất, đọc phản hồi và in thông báo nhận pong. Nhờ trình tự gửi–nhận này, thông báo ping luôn xuất hiện trước pong mà không cần quy định tiến trình nào phải được chạy trước. Các đầu pipe không sử dụng được đóng ở mỗi bên; những đầu còn lại cũng được đóng sau khi trao đổi xong.

Trình tự trao đổi giữa tiến trình cha và con được trình bày trong mã giả dưới đây, tương ứng với hàm main().

pingpong

```
1  PROCEDURE Main()
2      Create channels p (parent to child) and q (child to parent)
3      Create a child process
4      IF channel creation or process creation fails THEN
5          REPORT error; EXIT 1
6      END IF
7      IF executing in the parent THEN
8          Close unused channel ends
9          Send one byte through p; close the sending end
10         Wait for the child to finish
11         Receive one byte through q
12         OUTPUT parent identifier and "received pong"
13         Close the receiving end
14     ELSE
15         Close unused channel ends
16         Receive one byte through p
17         OUTPUT child identifier and "received ping"
18         Send the same byte through q
19         Close the remaining channel ends
20     END IF
21     On any send or receive failure, report error and exit 1
22     EXIT 0
23 END PROCEDURE
```

Nhận xét & khó khăn: *[Điền nếu có.]*

4.3 primes

Mục tiêu.

- **Chức năng:** Sinh và in các số nguyên tố trong phạm vi từ 2 đến 280.
- **Tham số:** Không nhận tham số dòng lệnh; phạm vi xử lý được cố định theo yêu cầu của bài thực hành.
- **Mục tiêu xử lý:** Thể hiện mô hình sàng Eratosthenes đồng thời và bảo đảm mỗi số nguyên tố được xuất đúng một lần.

Ý tưởng triển khai. Các số từ 2 đến 280 được đưa lần lượt qua một chuỗi tiến trình, mỗi tiến trình thực hiện một bước sàng. Bước đầu lấy 2 làm số nguyên tố và loại các số chia hết cho 2. Những số còn lại được chuyển sang bước sau, nơi 3 được chọn và các bội của 3 tiếp tục bị loại. Cách xử lý này lặp lại cho đến

khi không còn số cần xét. Số đầu tiên ở mỗi bước là số nguyên tố: nếu nó chia hết cho một số nguyên tố nhỏ hơn, nó đã bị loại ở bước sàng trước đó.

Các bước sàng nối với nhau bằng pipe và xử lý từng số khi nhận được, không cần gom toàn bộ dãy còn lại vào bộ nhớ. Tiến trình kế tiếp chỉ được tạo khi có số vượt qua bước lọc hiện tại. Sau khi gửi hết dữ liệu, tiến trình đóng đầu ghi để báo hết dữ liệu cho bước sau, rồi chờ bước đó hoàn tất trước khi thoát.

Mã giả dưới đây mô tả một bước sàng trong hàm primes(left). Các bước tiếp theo áp dụng cùng cách xử lý cho những số còn lại.

primes

```
1  PROCEDURE Primes(left)
2     prime ← ReceiveInteger(left)
3     IF a complete integer was not received THEN
4        Close left; EXIT 0
5     END IF
6     OUTPUT "prime" followed by prime
7     right ← no outgoing channel
8     WHILE a complete integer n can be received from left DO
9        IF n is divisible by prime THEN
10          CONTINUE
11       END IF
12       IF right has not been created THEN
13          Create a channel and a child process
14          IF creation fails THEN
15             REPORT error; EXIT 1
16          END IF
17          Child closes unused ends and runs Primes(new channel)
18          Parent keeps the sending end as right
19       END IF
20       Send n through right
21       IF sending fails THEN
22          REPORT error; EXIT 1
23       END IF
24    END WHILE
25    Close left
26    IF right exists THEN
27       Close right; wait for the child to finish
28    END IF
29    EXIT 0
30 END PROCEDURE
```

Quản lý Process & Pipe. Mỗi bước lọc chỉ giữ những kênh liên lạc cần thiết. Khi đã chuyển hết các số, bước đó báo cho bước tiếp theo biết để việc xử lý có thể kết thúc. Các tiến trình chờ phần việc phía sau hoàn tất trước khi thoát.

Nhận xét & khó khăn: *[Điền nếu có.]*

4.4 cp

Mục tiêu.

- **Chức năng:** Sao chép toàn bộ nội dung từ tệp nguồn src sang tệp đích dst.
- **Tham số:** Nhận đúng hai tham số đường dẫn theo cú pháp cp src dst.
- **Xử lý lỗi:** Kiểm tra nguồn không hợp lệ, đích không thể tạo hoặc mở, trường hợp nguồn và đích cùng trỏ tới một tệp, đồng thời báo lỗi khi cú pháp không đúng.

Ý tưởng triển khai. Trước khi sao chép, cần xác định nguồn và đích có phải là hai tệp khác nhau hay không. Chỉ so sánh đường dẫn là chưa đủ, bởi hai tên có thể cùng trỏ đến một tệp. Chương trình vì vậy kiểm tra thông tin nhận diện của tệp và loại tệp trước khi xóa nội dung đích. Nguồn phải là tệp thông thường; nếu đích đã tồn tại thì đích cũng phải đáp ứng điều kiện này.

Dữ liệu sau đó được đọc từ nguồn và ghi sang đích theo từng khối tối đa 512 byte. Mỗi lượt ghi đúng lượng dữ liệu vừa đọc được, nên phần cuối của tệp vẫn được sao chép đầy đủ dù ngắn hơn một khối. Việc sao chép tuần tự giữ nguyên thứ tự các byte và không đòi hỏi bộ nhớ tăng theo kích thước tệp. Khi gặp lỗi đọc hoặc ghi, chương trình dừng và báo lỗi; dữ liệu đã ghi vào đích trước đó không được khôi phục.

Quy trình kiểm tra tệp và sao chép dữ liệu trong hàm main() được tóm tắt bằng mã giả dưới đây.

cp

```
1  PROCEDURE Main(arguments)
2      Require exactly two arguments: source and destination
3      Open source and verify that it is a regular file
4      IF destination already exists THEN
5          Verify that it is a different regular file
6      END IF
7      IF an argument or file check fails THEN
8          Close opened files; report error; EXIT 1
9      END IF
10     Create or open destination, replacing its old contents
11     IF destination cannot be opened THEN
12         Close source; report error; EXIT 1
13     END IF
14     WHILE reading source yields a nonempty portion DO
15         Write exactly that portion to destination
16         IF the full portion was not written THEN
17             Close both files; report error; EXIT 1
18         END IF
```

```
    END WHILE
    IF the last read failed THEN
        Close both files; report error; EXIT 1
    END IF
    Close both files
    EXIT 0
END PROCEDURE
```

Nhận xét & khó khăn: *[Điền nếu có.]*

4.5 tree

Mục tiêu.

- **Chức năng:** Hiển thị thư mục và tệp tin theo dạng cây phân cấp.
- **Tham số:** Nhận đường dẫn tùy chọn path, mặc định là thư mục hiện hành .; hỗ trợ -L depth để giới hạn độ sâu và -d để chỉ liệt kê thư mục.
- **Xử lý lỗi:** Phát hiện và thông báo rõ ràng các đường dẫn, tham số hoặc tùy chọn không hợp lệ.

Ý tưởng triển khai. Từ thư mục được chọn, chương trình duyệt theo chiều sâu: in tên một mục, duyệt hết nội dung bên trong nếu đó là thư mục, rồi mới chuyển sang mục kế tiếp. Mức thụt lề tăng theo độ sâu, thể hiện vị trí của từng mục trong cây. Hai mục . và .. được bỏ qua để không quay lại thư mục hiện tại hoặc thư mục cha; các mục chưa sử dụng cũng không được đưa vào kết quả.

Tùy chọn -L giới hạn số tầng được hiển thị, còn -d chỉ giữ lại các thư mục. Khi đạt độ sâu cho phép, chương trình không duyệt sâu hơn nhánh đó. Nếu đường dẫn đầu vào là một tệp thay vì thư mục, kết quả chỉ gồm tên tệp, trừ trường hợp đang bật -d.

Mã giả dưới đây thể hiện cách duyệt và hiển thị một nhánh thư mục, dựa trên hàm tree_traverse(path, level).

tree

```
PROCEDURE TreeTraverse(path, level)
    IF a depth limit is set and level reaches it THEN
        RETURN
    END IF
    Open directory path
    IF opening fails THEN
        REPORT error; RETURN
    END IF
```

```
    IF child paths would exceed the allowed length THEN
      Close directory; report error; RETURN
    END IF
    FOR EACH entry read from directory DO
      IF entry is unused, "." or ".." THEN
        CONTINUE
      END IF
      childPath ← Join(path, entry.name)
      IF information for childPath cannot be obtained THEN
        REPORT error; CONTINUE
      END IF
      IF directories-only mode excludes entry THEN
        CONTINUE
      END IF
      Display entry.name with indentation for level + 1
      IF entry is a directory THEN
        TreeTraverse(childPath, level + 1)
      END IF
    END FOR
    Close directory
END PROCEDURE
```

Nhận xét & khó khăn: *[Điền nếu có.]*

4.6 du

Mục tiêu.

- **Chức năng:** Tính và hiển thị dung lượng sử dụng của tệp hoặc thư mục theo đơn vị byte.
- **Tham số:** Nhận đường dẫn tùy chọn path, mặc định là .; hỗ trợ -s để chỉ in tổng dung lượng và -a để hiển thị thêm dung lượng của từng tệp.
- **Đầu ra:** Hiển thị dung lượng theo byte cùng đường dẫn tương ứng và tuân theo chế độ hiển thị đã chọn.

Ý tưởng triển khai. Dung lượng của một thư mục được tính từ các tệp và thư mục con bên trong. Với tệp thông thường hoặc tệp thiết bị, chương trình lấy kích thước do hệ thống ghi nhận. Với thư mục con, chương trình tính xong tổng của nhánh đó rồi cộng vào thư mục cha. Kết quả vì vậy được tổng hợp từ dưới lên: các mục con hoàn tất trước, thư mục chứa chúng hoàn tất sau.

Các tùy chọn chỉ thay đổi phần kết quả được in, không thay đổi cách cộng dung lượng. Chế độ mặc định in tổng của từng thư mục; -a in thêm từng tệp, còn -s chỉ in tổng của đường dẫn được chọn. Trong bài này, dung lượng là tổng kích

thước theo byte, không tính kích thước bản thân thư mục hay số khối đĩa thực tế sử dụng. Nhánh không truy cập được sẽ được báo lỗi và bỏ qua khi tính tổng.

Cách tính dung lượng và cộng dồn kết quả từ các mục con được trình bày trong mã giả của hàm du(path) dưới đây.

du

```
1  PROCEDURE Du(path)
2      Obtain information about path
3      IF information cannot be obtained THEN
4          REPORT error; RETURN 0
5      END IF
6      IF path is a regular file or a device THEN
7          IF all-files mode is enabled and summary mode is off THEN
8              OUTPUT size in bytes and path
9          END IF
10         RETURN size in bytes
11     END IF
12     IF path is not a directory THEN
13         RETURN 0
14     END IF
15     Open directory and check the allowed child-path length
16     IF opening or the length check fails THEN
17         Close directory if opened; report error; RETURN 0
18     END IF
19     total ← 0
20     FOR EACH entry read from directory DO
21         IF entry is unused, "." or ".." THEN
22             CONTINUE
23         END IF
24         total ← total + Du(Join(path, entry.name))
25     END FOR
26     Close directory
27     IF summary mode is off THEN
28         OUTPUT total and path
29     END IF
30     RETURN total
31 END PROCEDURE
```

Nhận xét & khó khăn: *[Điền nếu có.]*

4.7 diff

Mục tiêu.

- **Chức năng:** So sánh nội dung của hai tệp theo từng dòng và xác định các vị trí khác biệt.

- **Tham số:** Nhận hai tệp bắt buộc file1, file2 và tùy chọn -q theo cú pháp diff file1 file2 [-q].
- **Đầu ra:** Ở chế độ thông thường, chỉ ra các dòng khác biệt của từng tệp; với -q, chỉ thông báo hai tệp có khác nhau hay không.

Ý tưởng triển khai. Hai tệp được đọc song song và so sánh các dòng có cùng số thứ tự. Ngoài nội dung, chương trình kiểm tra độ dài và dấu xuống dòng, đồng thời phân biệt dòng trống với trường hợp tệp đã hết dữ liệu. Do đó, hai dòng có cùng phần chữ nhưng khác dấu xuống dòng cuối tệp vẫn được xem là khác nhau. Bộ đệm được mở rộng theo độ dài dòng để không cắt mất phần nội dung cần so sánh.

Với mỗi khác biệt, kết quả ghi tên tệp, số dòng và nội dung của hai phía. Nếu một tệp kết thúc trước, các dòng còn lại ở tệp kia được đối chiếu với trạng thái hết tệp. Khi có -q, chương trình dừng ở khác biệt đầu tiên và chỉ in thông báo. Phép so sánh trong bài không căn chỉnh lại dòng khi có chèn hoặc xóa: một dòng thêm vào giữa tệp có thể khiến nhiều cặp dòng phía sau được báo khác nhau.

Mã giả dưới đây mô tả quy trình so sánh trong hàm main(), từ việc đọc từng cặp dòng đến ghi nhận và hiển thị khác biệt.

diff

```
1  PROCEDURE Main(arguments)
2      Parse two paths and optional quiet flag at either end
3      Reject invalid syntax or paths beginning with a hyphen
4      Open both files and verify that they are regular files
5      On a setup failure, close opened files and exit 2
6      status ← 0; number ← 1
7      LOOP
8          first ← ReadLine(firstFile)
9          second ← ReadLine(secondFile)
10         IF either line could not be read THEN
11             REPORT error; status ← 2; BREAK
12         END IF
13         IF neither file has another line THEN
14             BREAK
15         END IF
16         IF presence, length, newline or contents differ THEN
17             status ← 1
18             IF quiet mode is enabled THEN
19                 OUTPUT "diff: files differ"; BREAK
20             END IF
21             PrintLine(firstFile, number, first)
22             PrintLine(secondFile, number, second)
```

```
23          IF either output operation fails THEN
24            REPORT error; status ← 2; BREAK
25          END IF
26        END IF
27        number ← number + 1
28      END LOOP
29      Release stored lines and close both files
30      EXIT status
31  END PROCEDURE
```

Nhận xét & khó khăn: *[Điền nếu có.]*

5. Kết quả kiểm thử tổng hợp

5.1 Môi trường và cách thực hiện

Kiểm thử được thực hiện tự động trên xv6 chạy bằng QEMU 8.2.2, với 128 MB bộ nhớ và 3 CPU mô phỏng. Mã nguồn được biên dịch trên Ubuntu 24.04 trong WSL bằng GCC 13.3.0 cho RISC-V. Bộ chạy kiểm thử tạo một bản sao tạm của xv6, biên dịch lại chương trình và tạo ảnh hệ thống tệp riêng; vì vậy, các tệp thử nghiệm không làm thay đổi fs.img trong thư mục làm việc.

Bộ kiểm thử gồm cptest, difftest, linetest và coretest, đặt tại tests/lab01/. Các bài kiểm tra đối chiếu kết quả với giá trị mong đợi, bao gồm nội dung tệp, thông báo xuất ra và mã thoát. Riêng sleep, thời gian chờ được đo bằng bộ đếm tick của xv6. Lệnh chạy từ thư mục gốc của dự án, trong Ubuntu WSL, là:

```
python3 tests/lab01/run.py \
  --log docs/test-results/lab01-qemu.log
```

5.2 Kết quả theo từng chương trình

Lần chạy ghi nhận **77/77 phép kiểm tra đạt**, không xuất hiện thông báo FAIL và bộ chạy kiểm thử kết thúc với mã 0. Con số này bao gồm các lần chạy lặp lại và một kiểm tra tạo liên kết cứng phục vụ thử nghiệm cp; không phải số lượng trường hợp độc lập hay tỷ lệ bao phủ mã nguồn.

Đối tượng	Phạm vi kiểm tra	Kết quả ghi nhận	Đạt/tổng
sleep	Chờ 5 tick, 0 tick; thiếu, thừa, sai định dạng và số âm	Lượt sleep 5 đo được 5 tick; thông báo và mã thoát đúng kỳ vọng	6/6
pingpong	Chạy 3 lần; kiểm tra thứ tự thông báo và PID	Mỗi lần in ping trước pong; hai PID khác nhau; mã thoát 0	3/3
primes	Chạy 2 lần; đối chiếu dãy bằng phép thử chia độc lập	Đủ 59 số nguyên tố từ 2 đến 280, đúng thứ tự, không thừa hoặc trùng	2/2
cp	Tệp 0–4097 byte; ghi đè; cùng tệp, liên kết cứng; đối số và đường dẫn lỗi	Đúng từng byte và kích thước; giữ nguyên nguồn trong các ca từ chối sao chép	18/18
tree	Cây nhiều tầng; -d, -L 1, -L 2; tệp, thư mục rỗng và đầu vào lỗi	Cấu trúc, thụt lề, giới hạn độ sâu, thông báo và mã thoát đúng kỳ vọng	11/11
du	Tổng thư mục; -a, -s, kết hợp hai cờ; tệp đơn và đầu vào lỗi	Tệp mẫu 3, 5, 2 byte cho tổng 10 byte; kết quả chi tiết và tóm tắt đúng	8/8
diff	Tệp rỗng; dòng khác nhau; -q; hết tệp, dấu xuống dòng; dòng dài và byte 0	Nội dung xuất và mã thoát 0/1/2 đúng kỳ vọng, kể cả dòng 1200 byte	20/20
Đọc dòng	Dòng trống, dòng cuối, hết tệp, mở rộng bộ đệm, byte 0 và lỗi đọc	Phân biệt dòng trống với hết tệp; giữ đủ nội dung và nhận biết lỗi đọc	9/9
Tổng		**Tất cả phép kiểm tra đều đạt**	**77/77**

5.3 Nhật ký thực thi và phạm vi kết luận

Nhật ký QEMU đầy đủ được lưu tại docs/test-results/lab01-qemu.log. Trích các dòng xác nhận hoàn tất từ lần chạy thực tế:

```
ALL CP TESTS PASSED
ALL DIFF TESTS PASSED
ALL LINE TESTS PASSED
MEASURE: sleep 5 elapsed=5 ticks
ALL SLEEP TESTS PASSED
```

```
ALL PINGPONG TESTS PASSED
ALL PRIMES TESTS PASSED
ALL TREE TESTS PASSED
ALL DU TESTS PASSED
ALL CORE TESTS PASSED
```

Các kết quả trên xác nhận hành vi của chương trình trong những trường hợp đã kiểm tra, không chứng minh chương trình không còn lỗi. Bộ test chưa mô phỏng đầy đĩa, thiếu bộ nhớ, cạn tài nguyên tiến trình hoặc mọi giới hạn đường dẫn; mẫu sao chép lớn nhất là 4097 byte và mẫu dòng dài là 1200 byte. Thời gian 5 tick là số đo của lượt chạy này, không phải cam kết tiến trình luôn được lập lịch lại đúng thời điểm trên mọi lần chạy.

Đây là **bộ kiểm thử riêng cho phạm vi Lab 01 của báo cáo**, không phải kết quả make grade. Bộ chấm grade-lab-util có trong kho còn kiểm tra find và xargs, nhưng không bao gồm cp, tree, du, diff. Vì chưa chạy toàn bộ bộ chấm đó trong lần kiểm thử này, báo cáo không ghi điểm chấm MIT hoặc quy đổi 77/77 thành điểm trên 100.

6. Khó khăn gặp phải và giải pháp khắc phục

1. **Vấn đề 1: Tràn số lượng File Descriptor trong bài primes.**

   Nguyên nhân: Tiến trình con kế thừa toàn bộ bảng file descriptor của tiến trình cha nhưng không đóng kịp thời các đầu ghi của các stage trước đó, dẫn đến cạn kiệt tài nguyên (vượt quá NOFILE trong kernel).

   Giải pháp: Đóng ngay lập tức các đầu pipe không dùng ở cả cha và con trước khi bước vào vòng lặp xử lý dữ liệu.

2. **Vấn đề 2: Lặp vô hạn khi duyệt thư mục trong tree và du.**

   Nguyên nhân: Bỏ sót việc lọc hai liên kết đặc biệt là thư mục hiện hành (.) và thư mục cha (..).

   Giải pháp: Bổ sung kiểm tra strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0 trước khi gọi hàm đệ quy.

7. Kết luận và hướng phát triển

Qua bài thực hành Lab 01, nhóm đã hoàn thiện toàn diện các kỹ năng lập trình không gian người dùng trên hệ điều hành xv6:

- Nắm vững cơ chế vận hành của system call và sự chuyển giao giữa User Space và Kernel Space.
- Làm chủ kỹ thuật điều khiển luồng tiến trình và đồng bộ hóa thông qua các cơ chế IPC cơ bản nhưng hiệu quả.
- Hiểu rõ cấu trúc tổ chức và lưu trữ dữ liệu của hệ thống tệp tin xv6 ở mức độ nhị phân.

Tài liệu tham khảo

[1] Russ Cox, Frans Kaashoek, Robert Morris, *xv6: a simple, Unix-like teaching operating system*, bản RISC-V, ngày 31/08/2024. Tham khảo chương 1 về system call và pipe, chương 8 về hệ thống tệp. https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf.

[2] MIT, học phần 6.1810, *Lab: Xv6 and Unix utilities*, Fall 2024. Yêu cầu và hướng dẫn cho sleep, pingpong, primes. https://pdos.csail.mit.edu/6.1810/2024/labs/util.html.

[3] Tài liệu hướng dẫn thực hành Hệ điều hành Lab 01 do Giảng viên bộ môn biên soạn.

[4] MIT PDOS, *xv6-riscv*, kho mã nguồn chính thức. Tham khảo cách tổ chức tiến trình, pipe, hệ thống tệp và chương trình ls. Mã nguồn thực hiện bài được đối chiếu trực tiếp trong thư mục xv6-labs-2024/ của dự án. https://github.com/mit-pdos/xv6-riscv.

[5] Russ Cox, *Bell Labs and CSP Threads*. Tham khảo mô hình sàng số nguyên tố bằng các tiến trình nối với nhau qua pipe. https://swtch.com/~rsc/thread/.

[6] MIT, học phần 6.1810, *Tools*, Fall 2024. Hướng dẫn chuẩn bị công cụ RISC-V, QEMU và môi trường chạy xv6. https://pdos.csail.mit.edu/6.1810/2024/tools.html.

[7] GNU Project, *GNU Coreutils Manual*, các mục *cp invocation* và *du invocation*. Tham khảo chức năng và cách sử dụng các tiện ích sao chép, thống kê dung lượng trên Unix. https://www.gnu.org/software/coreutils/manual/coreutils.html.

[8] GNU Project, *Comparing and Merging Files*, mục *Invoking diff*. Tham khảo cách diễn giải kết quả so sánh và mã thoát của diff. https://www.gnu.org/software/diffutils/manual/html_node/Invoking-diff.html.

Các tài liệu GNU được dùng để tham khảo hành vi của tiện ích Unix; các chương trình trong bài chỉ thực hiện phạm vi yêu cầu của Lab 01, không có đầy đủ tính năng của phiên bản GNU.